# VISVESVARAYA TECHNOLOGICAL UNIVERSITY

**"JnanaSangama", Belgaum -590014, Karnataka.**



**LAB REPORT**

**On**

**ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)**

**Submitted by**

**Manasvi Singh (1BF24CS164)**

**in partial fulfillment for the award of the degree of**
**BACHELOR OF ENGINEERING**
**in**
**COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING**
**(Autonomous Institution under VTU)**
**BENGALURU-560019**
**February to June 2026**

# CERTIFICATE

**B. M. S. College of Engineering,**
**Bull Temple Road, Bangalore 560019**
**(Affiliated To Visvesvaraya Technological University, Belgaum)**
**Department of Computer Science and Engineering**



This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Manasvi Singh **(1BF24CS164)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.


**Prof. Manjula S**
Assistant Professor
Department of CSE
BMSCE, Bengaluru

**Dr. Kavitha Sooda**
Professor and Head
Department of CSE
BMSCE, Bengaluru

# Index Sheet


| Sl. No. | Experiment Title | Page No. |
|---|---|---|
| 1 | Write program to obtain the Topological ordering of vertices in a given digraph.<br>LeetCode exercises on topological sorting. | 4-6<br>7 |
| 2 | Implement Johnson Trotter algorithm to generate permutations. | 8-10 |
| 3 | Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.<br>LeetCode Program related to sorting. | 11-12<br>13 |
| 4 | Sort a given set of N integer elements using Quick Sort technique and compute its time taken.<br>LeetCode Program related to sorting. | 14-15<br>16 |
| 5 | Sort a given set of N integer elements using Heap Sort technique and compute its time taken. | 17-19 |
| 6 | Implement 0/1 Knapsack problem using dynamic programming.<br>LeetCode Program related to Knapsack problem or Dynamic Programming. | 20-21<br>22 |
| 7 | Implement All Pair Shortest paths problem using Floyd's algorithm.<br>LeetCode Program related to shortest distance calculation. | 23-24<br>25 |
| 8 | Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.<br>Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm. | 26-32 |
| 9 | Implement Fractional Knapsack using Greedy technique.<br>LeetCode Program related to Greedy Technique algorithms. | 33-35<br>36 |
| 10 | From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm. | 37-38 |
| 11 | Implement "N-Queens Problem" using Backtracking. | 39-40 |


**Course outcomes:**

| | |
|---|---|
| CO1 | Analyze time complexity of algorithms using asymptotic notations. |
| CO2 | Apply various algorithm design techniques for the given problem. |
| CO3 | Evaluate the appropriate algorithm/design technique for solving a given problem. |
| CO4 | Conduct practical experiments by applying appropriate algorithm design technique to solve problems. |

## Lab program 1:

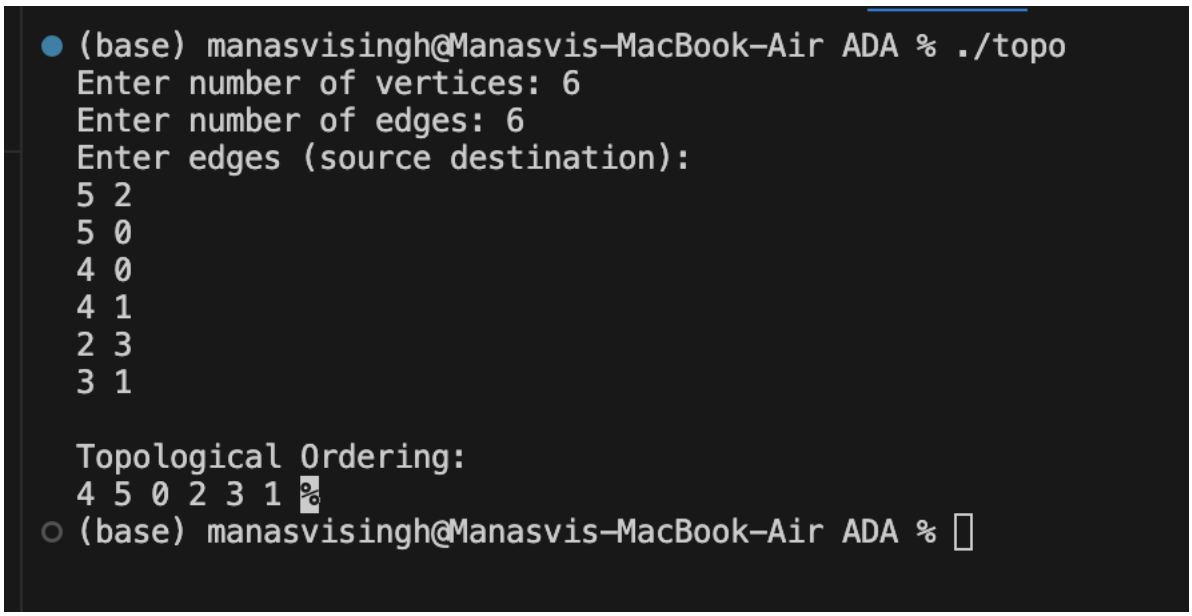
**Complete program in C**

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int graph[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int front = 0, rear = 0;
void enqueue(int vertex) {
   queue[rear++] = vertex;
}
int dequeue() {
   return queue[front++];
}
void topologicalSort(int vertices) {
   for (int i = 0; i < vertices; i++) {
      indegree[i] = 0;
      for (int j = 0; j < vertices; j++) {
         if (graph[j][i] == 1)
            indegree[i]++;
      }
   }
   // Insert all vertices with indegree 0 into queue
   for (int i = 0; i < vertices; i++) {
      if (indegree[i] == 0)
         enqueue(i);
   }
   int count = 0;
```

```
    printf("\nTopological Ordering:\n");
    while (front < rear) {
        int vertex = dequeue();
        printf("%d ", vertex);
        count++;
        for (int i = 0; i < vertices; i++) {
            if (graph[vertex][i] == 1) {
                indegree[i]--;
                if (indegree[i] == 0)
                    enqueue(i);
            }
        }
    }
    if (count != vertices) {
        printf("\nGraph contains a cycle!");
    }
}
int main() {
    int vertices, edges;
    printf("Enter number of vertices: ");
    scanf("%d", &vertices);
    printf("Enter number of edges: ");
    scanf("%d", &edges);
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            graph[i][j] = 0;
        }
    }
    printf("Enter edges (source destination):\n");
```

```
    for (int i = 0; i < edges; i++) {
        int src, dest;
        scanf("%d %d", &src, &dest);
        graph[src][dest] = 1;
    }
    topologicalSort(vertices);
    return 0;
}
```

**Output:**



## LeetCode problem 207: Course Schedule

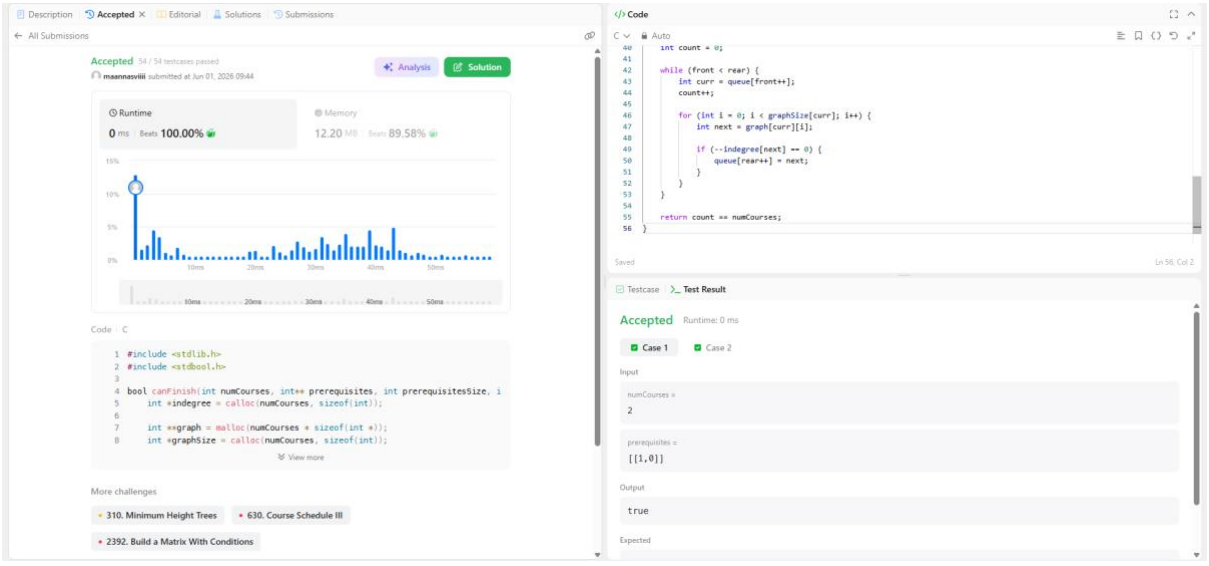
There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1.
You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that
you must take course bi first if you want to take course ai.
For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.
Return true if you can finish all courses. Otherwise, return false.

Example 1:
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is possible.

Example 2:
Input: numCourses = 2, prerequisites = [[1,0],[0,1]]
Output: false
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- 1 <= numCourses <= 2000
- 0 <= prerequisites.length <= 5000
- prerequisites[i].length == 2
- 0 <= ai, bi < numCourses
- All the pairs prerequisites[i] are unique



## Lab program 2:

**Complete program in C**

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
void swapDir(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
void printPermutation(int perm[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", perm[i]);
    printf("\n");
}
int getMobile(int perm[], int dir[], int n) {
    int mobile = 0;
    int mobileIndex = -1;
    for (int i = 0; i < n; i++) {
        if (dir[i] == LEFT && i != 0) {
            if (perm[i] > perm[i - 1] &&
                perm[i] > mobile) {
                mobile = perm[i];
                mobileIndex = i;
            }
```

```
        }
        if (dir[i] == RIGHT && i != n - 1) {
            if (perm[i] > perm[i + 1] &&
                perm[i] > mobile) {
                mobile = perm[i];
                mobileIndex = i;
        }}}
    return mobileIndex;
}
void johnsonTrotter(int n) {
    int perm[n];
    int dir[n];
    for (int i = 0; i < n; i++) {
        perm[i] = i + 1;
        dir[i] = LEFT;
    }
    printPermutation(perm, n);
    while (1) {
        int mobileIndex = getMobile(perm, dir, n);
        if (mobileIndex == -1)
            break;
        int swapIndex;
        if (dir[mobileIndex] == LEFT)
            swapIndex = mobileIndex - 1;
        else
            swapIndex = mobileIndex + 1;
        swap(&perm[mobileIndex], &perm[swapIndex]);
        swapDir(&dir[mobileIndex], &dir[swapIndex]);
        mobileIndex = swapIndex;
```

```
        for (int i = 0; i < n; i++) {
            if (perm[i] > perm[mobileIndex]) {
                dir[i] = -dir[i];
            }}
        printPermutation(perm, n);
    }}
int main() {
    int n;
    printf("Enter value of n: ");
    scanf("%d", &n);
    printf("\nPermutations using Johnson-Trotter Algorithm:\n");
    johnsonTrotter(n);
    return 0;
}
```

**Output:**

```
(base) manasvisingh@Manasvis-MacBook-Air ADA % gcc trotter.c -o trotter
(base) manasvisingh@Manasvis-MacBook-Air ADA % ./trotter
Enter value of n: 4

Permutations using Johnson-Trotter Algorithm:
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4
```

## Lab program 3:

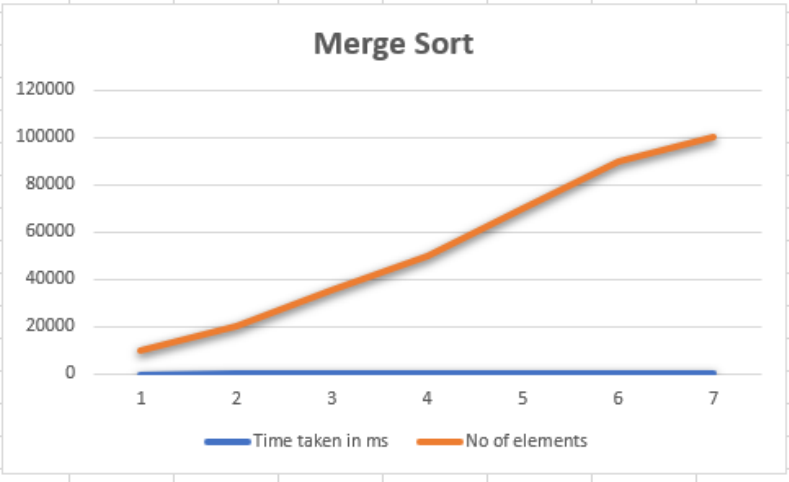
**Complete program in C**

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int c[100000];
void merge(int a[], int low, int mid, int high) {
   int i = low, j = mid + 1, k = low;
   while (i <= mid && j <= high) {
      if (a[i] < a[j])
         c[k++] = a[i++];
      else
         c[k++] = a[j++];
   }
   while (i <= mid)
      c[k++] = a[i++];
   while (j <= high)
      c[k++] = a[j++];
   for (i = low; i <= high; i++) {
      a[i] = c[i];
   }
}
void mergeSort(int a[], int low, int high) {
   if (low < high) {
      int mid = (low + high) / 2;
      mergeSort(a, low, mid);
      mergeSort(a, mid + 1, high);
      merge(a, low, mid, high);
   }
}
int main() {
   int a[100000], n, i;
   clock_t start, end;
```

```
    printf("No. of Elements: ");
    scanf("%d", &n);
    srand(time(NULL));
    for (i = 0; i < n; i++) {
        a[i] = rand() % 100000;
    }
    printf("\n");
    start = clock();
    mergeSort(a, 0, n - 1);
    end = clock();
    double time = (double)((end - start)*1000.0) / CLOCKS_PER_SEC;
    printf("\nTime taken: %f ms\n", time);
    return 0;
}
```

**Output:**

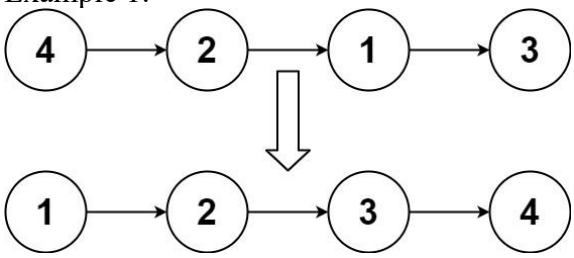
| No of elements | Time taken in ms |
|---|---|
| 10000 | 1 |
| 20000 | 2 |
| 35000 | 3 |
| 50000 | 4 |
| 70000 | 6 |
| 90000 | 8 |
| 100000 | 8 |



## LeetCode problem 148: Sort List

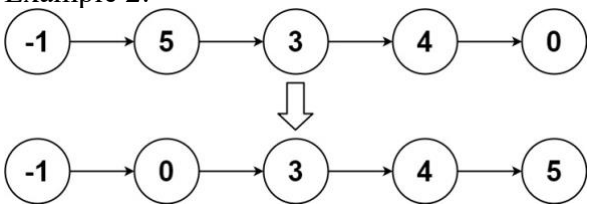
Given the head of a linked list, return *the list after sorting it in ascending order*.

Example 1:




Input: head = [4,2,1,3]
Output: [1,2,3,4]

Example 2:

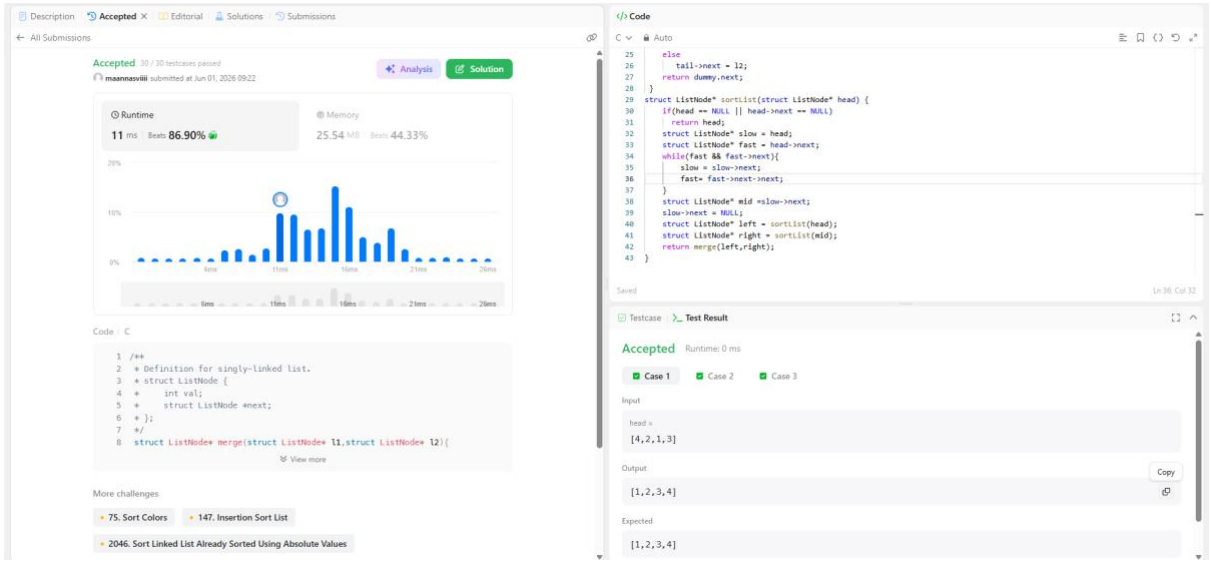



Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]

Example 3:
Input: head = []
Output: []

Constraints:

- The number of nodes in the list is in the range [0, 5 * 104].
- -105 <= Node.val <= 105



## Lab program 4:

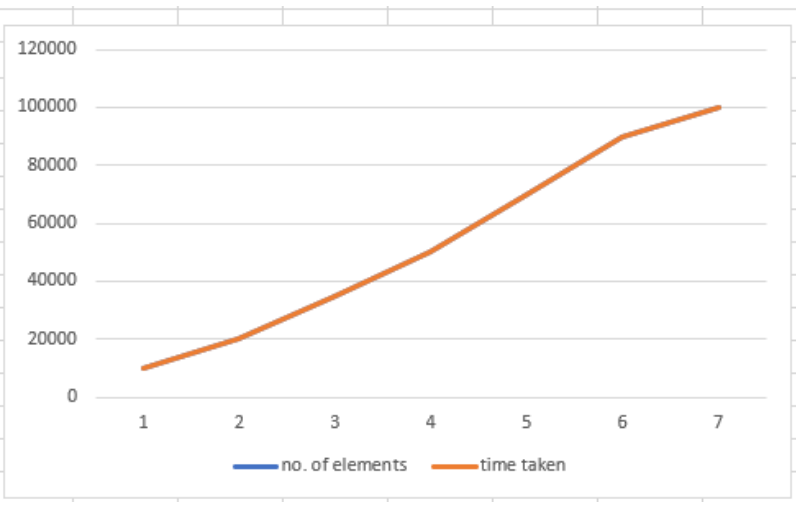
**Complete program in C**

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void swap(int *a, int *b) {
int temp = *a;
*a = *b;
*b = temp;
}
int partition(int arr[], int low, int high) {
int pivot = arr[high];
int i = low - 1;
for (int j = low; j < high; j++) {
if (arr[j] < pivot) {
i++;
swap(&arr[i], &arr[j]);
}
}
swap(&arr[i + 1], &arr[high]);
return i + 1;
}
void quickSort(int arr[], int low, int high) {
if (low < high) {
int pi = partition(arr, low, high);
quickSort(arr, low, pi - 1);
quickSort(arr, pi + 1, high);
}
}
```

```
int main() {
int n;
printf("Enter number of elements: ");
scanf("%d", &n);
int arr[n];
//srand(time(0));
for (int i = 0; i < n; i++) {
arr[i] = rand() % 1000;
}
clock_t start = clock();
quickSort(arr, 0, n - 1);
clock_t end = clock();
double time_taken = ((double)((end- start)*1000)) / CLOCKS_PER_SEC;
printf("\n\nTime taken: %f ms\n", time_taken);
return 0;
}
```

**Output:**

| Quick Sort | |
|---|---|
| no. of elements | time taken |
| 10000 | 1 |
| 20000 | 2 |
| 35000 | 2 |
| 50000 | 4 |
| 70000 | 6 |
| 90000 | 6 |
| 100000 | 8 |



## LeetCode problem 268: Missing Number

Given an array nums containing n distinct numbers in the range [0, n], return *the only number in the range that is missing from the array.*

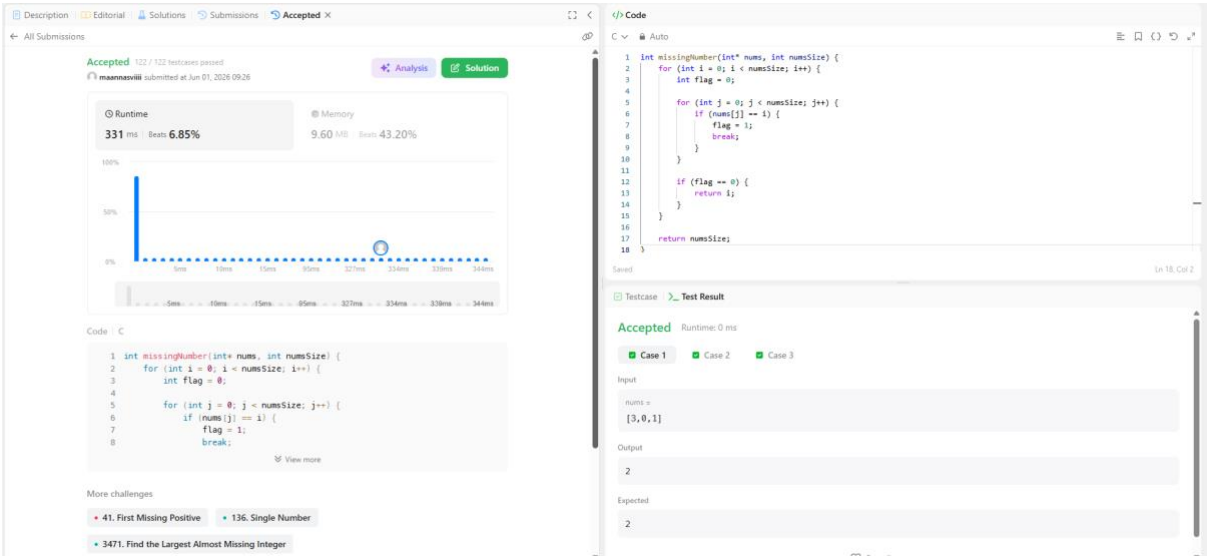
Example 1:
Input: nums = [3,0,1]
Output: 2
Explanation: n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.

Example 2:
Input: nums = [0,1]
Output: 2
Explanation: n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.

Example 3:
Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
Explanation: n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.

Constraints:

- n == nums.length
- 1 <= n <= 104
- 0 <= nums[i] <= n
- All the numbers of nums are unique.



## Lab program 5:

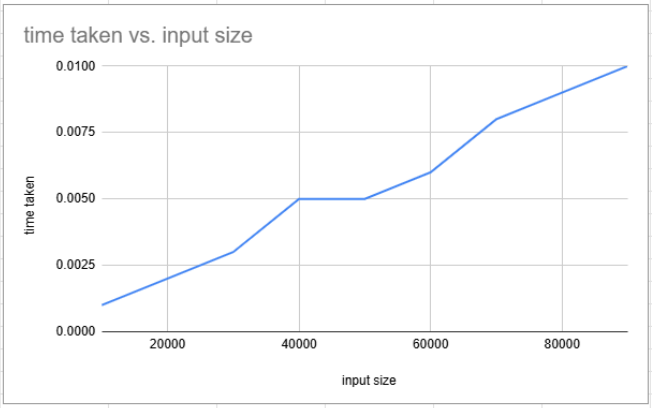
**Complete program in C**

```
#include <stdio.h>
#include <time.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
void heapify(int arr[], int n, int i) {
    int largest = i;        // Initialize largest as root
    int left = 2 * i + 1;   // Left child
    int right = 2 * i + 2;  // Right child
    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
```

```
}
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d integer elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    clock_t start = clock();
    heapSort(arr, n);
    clock_t end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array using Heap Sort:\n");
    printArray(arr, n);
    printf("\nTime taken = %f seconds\n", time_taken);
    return 0;
}
```

## Output:

| input size | time taken |
|---|---|
| 10000 | 0.001 |
| 20000 | 0.002 |
| 30000 | 0.003 |
| 40000 | 0.005 |
| 50000 | 0.005 |
| 60000 | 0.006 |
| 70000 | 0.008 |
| 80000 | 0.009 |
| 90000 | 0.01 |



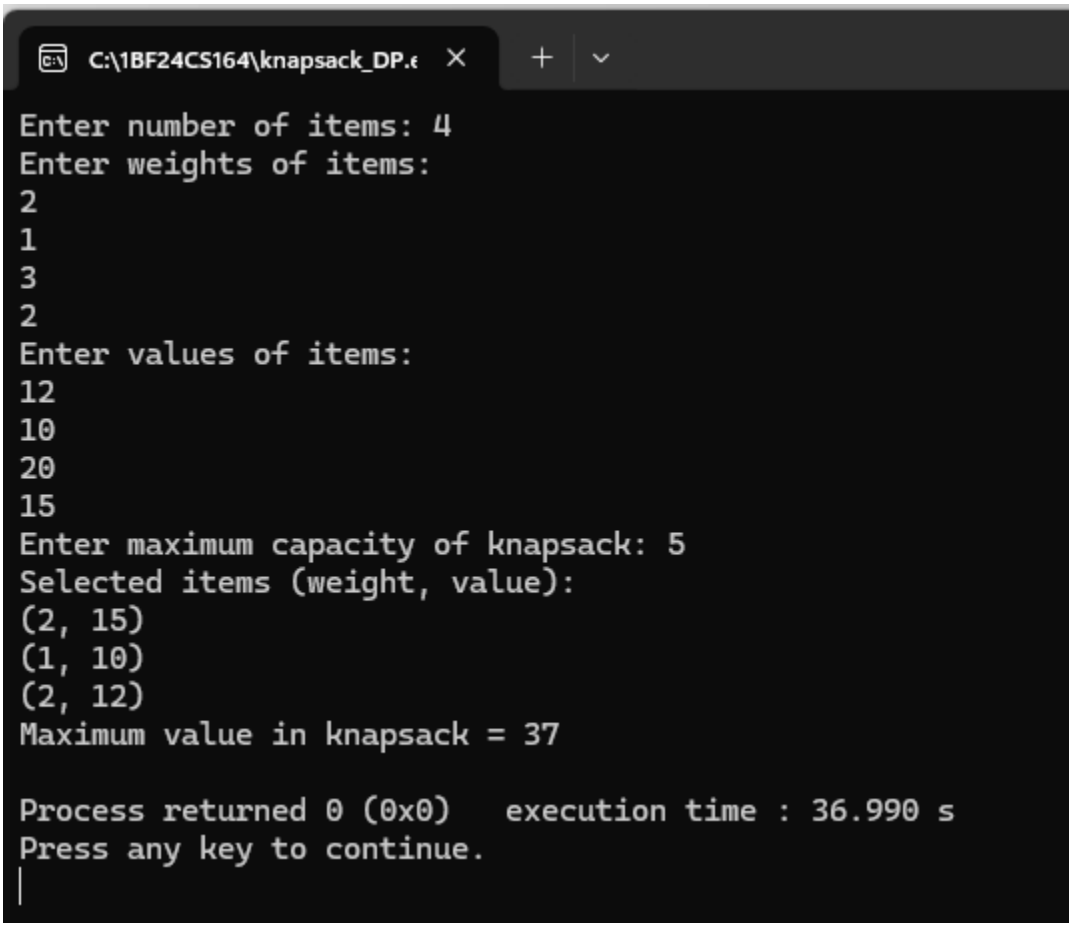
## Lab program 6:

**Complete program in C**

```
#include <stdio.h>
int max(int a, int b) {
return (a > b) ? a : b;
}
int knapsack(int W, int wt[], int val[], int n) {
int i, w;
int dp[n + 1][W + 1];
for (i = 0; i <= n; i++) {
for (w = 0; w <= W; w++) {
if (i == 0 || w == 0)
dp[i][w] = 0;
else if (wt[i - 1] <= w)
dp[i][w] = max( val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w] );
else
dp[i][w] = dp[i - 1][w];}}
printf("Selected items (weight, value):\n");
w = W;
for (i = n; i > 0 && w > 0; i--) {
if (dp[i][w] != dp[i - 1][w]) {
printf("(%d, %d)\n", wt[i - 1], val[i - 1]);
w -= wt[i - 1];}}
return dp[n][W];
}
int main() {
int n, W;
printf("Enter number of items: ");
scanf("%d", &n);
```

```
int wt[n], val[n];
printf("Enter weights of items:\n");
for (int i = 0; i < n; i++)
scanf("%d", &wt[i]);
printf("Enter values of items:\n");
for (int i = 0; i < n; i++)
scanf("%d", &val[i]);
printf("Enter maximum capacity of knapsack: ");
scanf("%d", &W);
int result = knapsack(W, wt, val, n);
printf("Maximum value in knapsack = %d\n", result);
return 0;
}
```

**Output:**



## LeetCode problem 70: Climbing Stairs

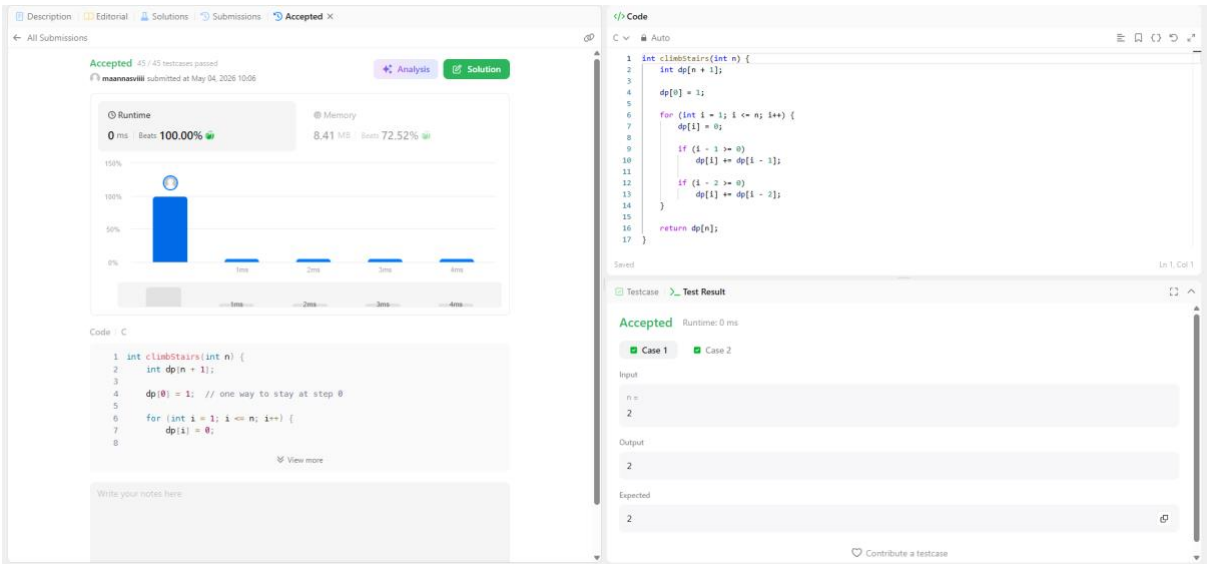
You are climbing a staircase. It takes n steps to reach the top.
Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:
Input: n = 2
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps

Example 2:
Input: n = 3
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Constraints:

- 1 <= n <= 45



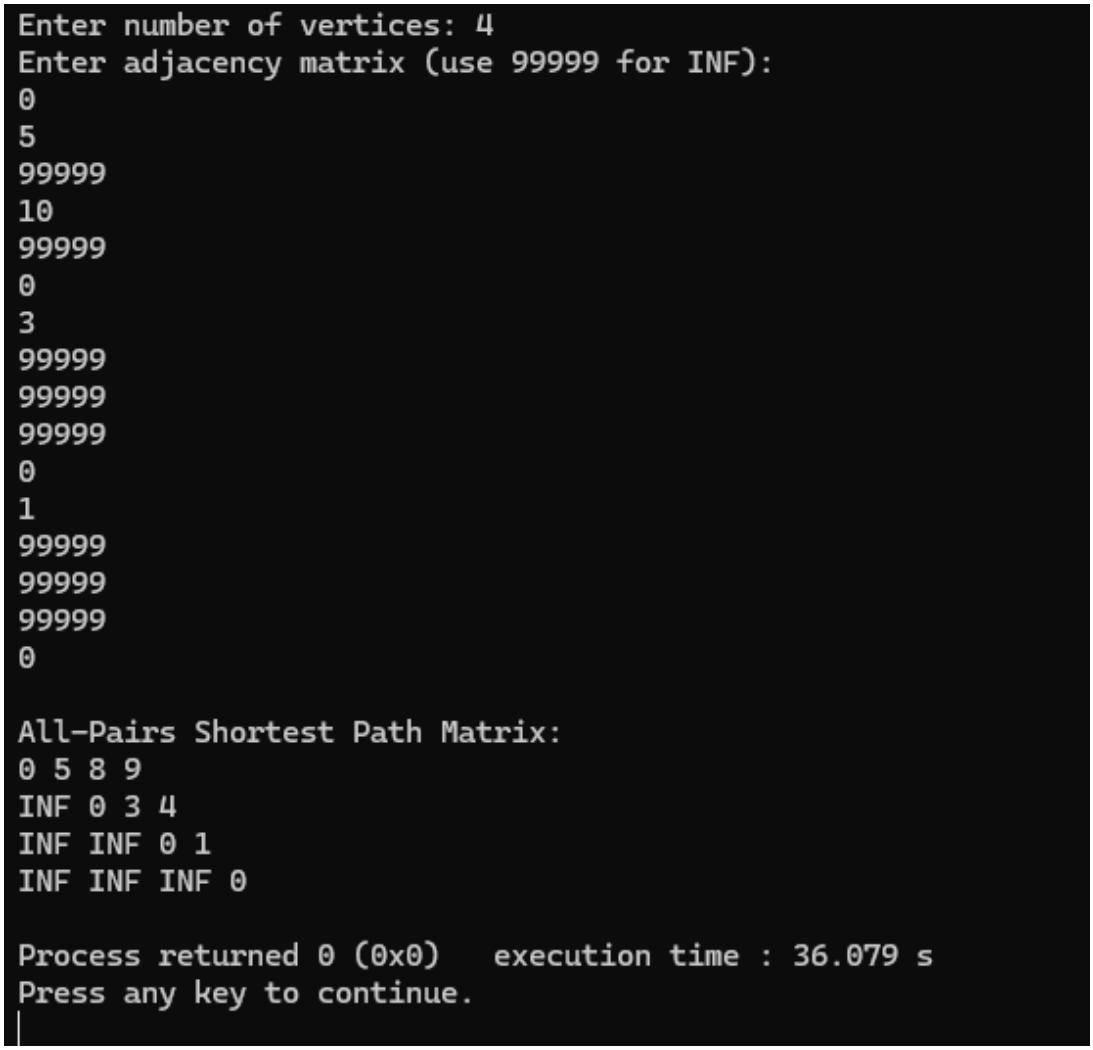
## Lab program 7:

**Complete program in C**

```
#include <stdio.h>
#define MAX 100
#define INF 99999
void floydWarshall(int graph[MAX][MAX], int n) {
int dist[MAX][MAX];
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
dist[i][j] = graph[i][j];}}
for (int k = 0; k < n; k++) {
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
if (dist[i][k] + dist[k][j] < dist[i][j]) {
dist[i][j] = dist[i][k] + dist[k][j];
}}}}
printf("\nAll-Pairs Shortest Path Matrix:\n");
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
if (dist[i][j] == INF)
printf("INF ");
else
printf("%d ", dist[i][j]);
}
printf("\n");
}}
int main() {
int n;
int graph[MAX][MAX];
```

```
printf("Enter number of vertices: ");
scanf("%d", &n);
printf("Enter adjacency matrix (use %d for INF):\n", INF);
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
scanf("%d", &graph[i][j]);
}
}
floydWarshall(graph, n);
return 0;
}
```

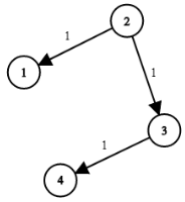
**Output:**



## LeetCode problem 743: Network Delay Time

You are given a network of n nodes, labeled from 1 to n. You are also given times, a list of travel times as directed edges times[i] = (ui, vi, wi), where ui is the source node, vi is the target node, and wi is the time it takes for a signal to travel from source to target.
We will send a signal from a given node k. Return *the minimum time it takes for all the* n *nodes to receive the signal*. If it is impossible for all the n nodes to receive the signal, return -1.

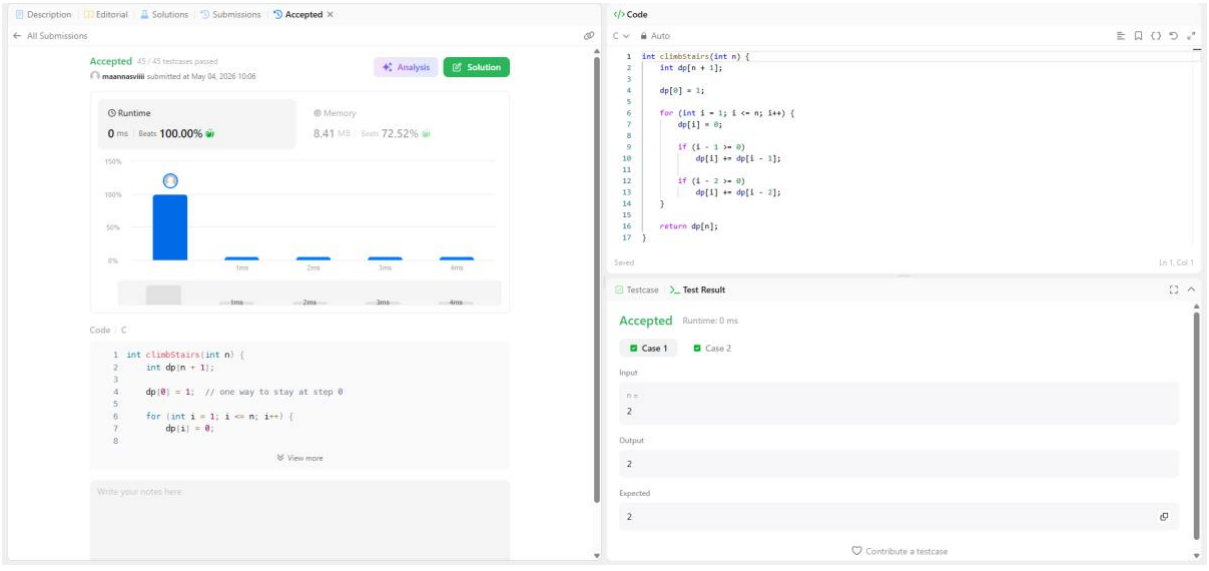
Example 1:




Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
Output: 2
Example 2:
Input: times = [[1,2,1]], n = 2, k = 1
Output: 1
Example 3:
Input: times = [[1,2,1]], n = 2, k = 2
Output: -1

Constraints:

- 1 <= k <= n <= 100
- 1 <= times.length <= 6000
- times[i].length == 3
- 1 <= ui, vi <= n
- ui != vi
- 0 <= wi <= 100
- All the pairs (ui, vi) are unique. (i.e., no multiple edges.)



## Lab program 8a:

**Complete program in C**

```
#include <stdio.h>
#define MAX 100
#define INF 999999
int graph[MAX][MAX];
int visited[MAX];
int findMinVertex(int key[], int visited[], int vertices) {
    int min = INF;
    int minIndex = -1;
    for (int i = 0; i < vertices; i++) {
            if (!visited[i] && key[i] < min) {
            min = key[i];
            minIndex = i;
        }
    }
    return minIndex;
}
void prims(int vertices) {
    int parent[MAX];
    int key[MAX];
    for (int i = 0; i < vertices; i++) {
        key[i] = INF;
        visited[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < vertices - 1; count++) {
        int u = findMinVertex(key, visited, vertices);
```

```
        visited[u] = 1;
        for (int v = 0; v < vertices; v++) {
            if (graph[u][v] &&
                !visited[v] &&
                graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
    int totalCost = 0;
    printf("\nEdges in Minimum Spanning Tree:\n");
    for (int i = 1; i < vertices; i++) {
        printf("%d - %d \tWeight = %d\n",
               parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }
    printf("\nMinimum Cost of Spanning Tree = %d\n", totalCost);
}
int main() {
    int vertices, edges;
    printf("Enter number of vertices: ");
    scanf("%d", &vertices);
    printf("Enter number of edges: ");
    scanf("%d", &edges);
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            graph[i][j] = 0;
        }
```

```
    }
    printf("Enter edges and weights:\n");
    printf("(source destination weight)\n");
    for (int i = 0; i < edges; i++) {
        int src, dest, weight;
        scanf("%d %d %d", &src, &dest, &weight);
        graph[src][dest] = weight;
        graph[dest][src] = weight; // Undirected graph
    }
    prims(vertices);
    return 0;
}
```

**Output:**



## Lab program 8b:

**Complete program in C**

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
struct Edge {
   int src, dest, weight;
};
struct Subset {
   int parent;
   int rank;
};
int compare(const void *a, const void *b) {
   return ((struct Edge *)a)->weight -
          ((struct Edge *)b)->weight;
}
int find(struct Subset subsets[], int i) {
   if (subsets[i].parent != i)
      subsets[i].parent =
         find(subsets, subsets[i].parent);
   return subsets[i].parent;
}
void Union(struct Subset subsets[],
          int x, int y) {
   int rootX = find(subsets, x);
   int rootY = find(subsets, y);
   if (subsets[rootX].rank <
      subsets[rootY].rank)
      subsets[rootX].parent = rootY;
```

```
    else if (subsets[rootX].rank >
             subsets[rootY].rank)
        subsets[rootY].parent = rootX;
    else {
        subsets[rootY].parent = rootX;
        subsets[rootX].rank++;
    }}
void kruskal(struct Edge edges[],
             int vertices,
             int edgeCount) {
    struct Edge result[MAX];
    int e = 0;
    int i = 0;
    qsort(edges, edgeCount,
          sizeof(edges[0]), compare);
    struct Subset subsets[MAX];
    for (int v = 0; v < vertices; v++) {
        subsets[v].parent = v;
        subsets[v].rank = 0;
    }
    while (e < vertices - 1 && i < edgeCount) {
        struct Edge nextEdge = edges[i++];
        int x = find(subsets, nextEdge.src);
        int y = find(subsets, nextEdge.dest);
        if (x != y) {
            result[e++] = nextEdge;
            Union(subsets, x, y);
        }
    }
```

```
    int totalCost = 0;
    printf("\nEdges in Minimum Spanning Tree:\n");
    for (i = 0; i < e; i++) {
        printf("%d -- %d == %d\n",
               result[i].src,
               result[i].dest,
               result[i].weight);
        totalCost += result[i].weight;
    }
    printf("\nMinimum Cost of Spanning Tree = %d\n", totalCost);
}
int main() {
    int vertices, edgeCount;
    printf("Enter number of vertices: ");
    scanf("%d", &vertices);
    printf("Enter number of edges: ");
    scanf("%d", &edgeCount);
    struct Edge edges[MAX];
    printf("Enter edges and weights:\n");
    printf("(source destination weight)\n");
    for (int i = 0; i < edgeCount; i++) {
        scanf("%d %d %d",
              &edges[i].src,
              &edges[i].dest,
              &edges[i].weight);
    }
    kruskal(edges, vertices, edgeCount);
    return 0;
}
```

**Output:**

```
PS C:\Program Files\CodeBlocks\MinGW\bin> C:\1BF24CS164\Krushkal.exe
Enter number of vertices: 5
Enter the cost adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in Minimum Spanning Tree:
1 Edge (0 -> 1) = 2
2 Edge (1 -> 2) = 3
3 Edge (1 -> 4) = 5
4 Edge (0 -> 3) = 6

Minimum Cost = 16
PS C:\Program Files\CodeBlocks\MinGW\bin>
```

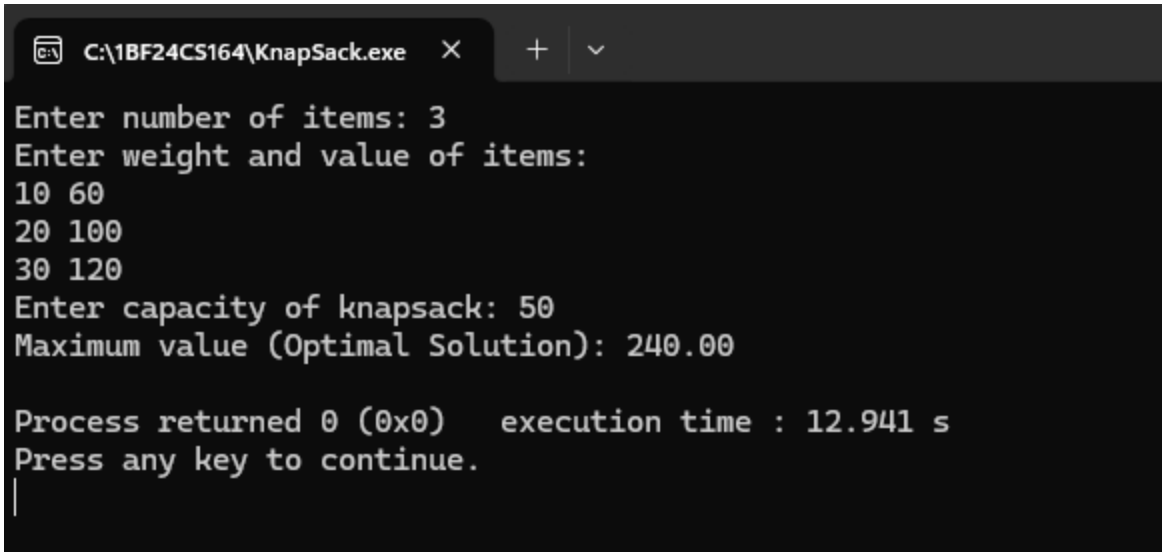
## Lab program 9:

**Complete program in C**

```
#include <stdio.h>
struct Item {
int weight;
int value;
float ratio;
};
void sortItems(struct Item items[], int n) {
for(int i = 0; i < n - 1; i++) {
for(int j = i + 1; j < n; j++) {
if(items[i].ratio < items[j].ratio) {
struct Item temp = items[i];
items[i] = items[j];
items[j] = temp;
}}}}
float fractionalKnapsack(int capacity, struct Item items[], int n) {
float totalValue = 0.0;
for(int i = 0; i < n; i++) {
if(capacity >= items[i].weight) {
capacity -= items[i].weight;
totalValue += items[i].value;
} else {
totalValue += items[i].ratio * capacity;
break;
}}
return totalValue;
}
int main() {
```

```
int n, capacity;
printf("Enter number of items: ");
scanf("%d", &n);
struct Item items[n];
printf("Enter weight and value of items:\n");
for(int i = 0; i < n; i++) {
scanf("%d %d", &items[i].weight, &items[i].value);
items[i].ratio = (float)items[i].value / items[i].weight;
}
printf("Enter capacity of knapsack: ");
scanf("%d", &capacity);
sortItems(items, n);
float maxValue = fractionalKnapsack(capacity, items, n);
printf("Maximum value (Optimal Solution): %.2f\n", maxValue);
return 0;
}
```

**Output:**



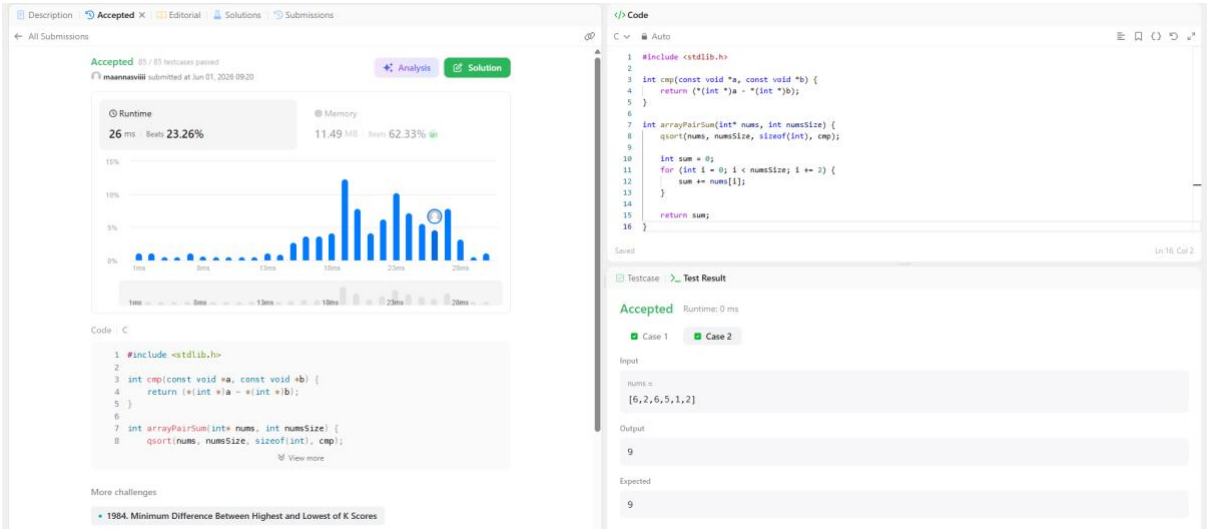
## LeetCode problem 561: Array Partition

Given an integer array nums of 2n integers, group these integers into n pairs (a1, b1), (a2, b2), ..., (an, bn) such that the sum of min(ai, bi) for all i is maximized. Return *the maximized sum*.

Example 1:
Input: nums = [1,4,3,2]
Output: 4
Explanation: All possible pairings (ignoring the ordering of elements) are:
1. (1, 4), (2, 3) -> min(1, 4) + min(2, 3) = 1 + 2 = 3
2. (1, 3), (2, 4) -> min(1, 3) + min(2, 4) = 1 + 2 = 3
3. (1, 2), (3, 4) -> min(1, 2) + min(3, 4) = 1 + 3 = 4
So the maximum possible sum is 4.

Example 2:
Input: nums = [6,2,6,5,1,2]
Output: 9
Explanation: The optimal pairing is (2, 1), (2, 5), (6, 6). min(2, 1) + min(2, 5) + min(6, 6) = 1 + 2 + 6 = 9.

Constraints:
- 1 <= n <= 104
- nums.length == 2 * n
- -104 <= nums[i] <= 104



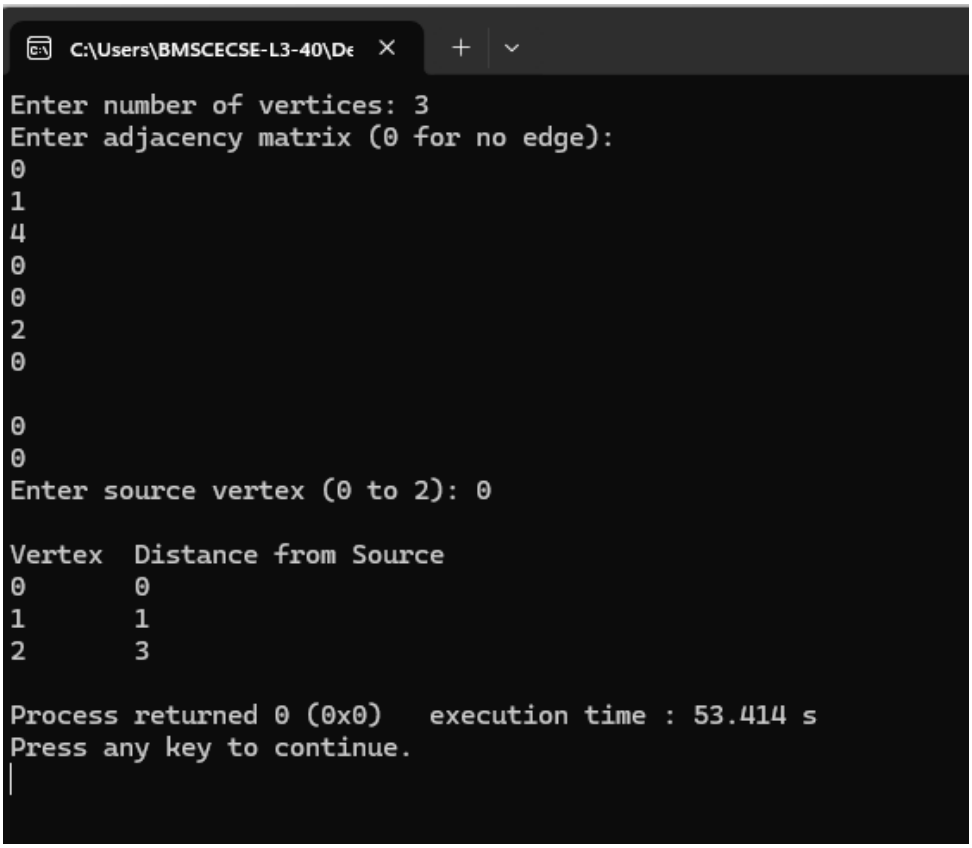
## Lab program 10:

**Complete program in C**

```
#include <stdio.h>
#include <limits.h>
#define MAX 100
int minDistance(int dist[], int visited[], int n) {
int min = INT_MAX, min_index = -1;
for (int v = 0; v < n; v++) {
if (visited[v] == 0 && dist[v] <= min) {
min = dist[v];
min_index = v;
}
}
return min_index;
}
void dijkstra(int graph[MAX][MAX], int n, int source) {
int dist[MAX];
int visited[MAX];
for (int i = 0; i < n; i++) {
dist[i] = INT_MAX;
visited[i] = 0;
}
dist[source] = 0;
for (int count = 0; count < n - 1; count++) {
int u = minDistance(dist, visited, n);
visited[u] = 1;
for (int v = 0; v < n; v++) {
if (!visited[v] &&
graph[u][v] &&
```

```
dist[u] != INT_MAX &&
dist[u] + graph[u][v] < dist[v]) {
dist[v] = dist[u] + graph[u][v];
}
}
}
printf("\nVertex\tDistance from Source\n");
for (int i = 0; i < n; i++) {
printf("%d\t%d\n", i, dist[i]);
}
}
int main() {
int n, source;
int graph[MAX][MAX];
printf("Enter number of vertices: ");
scanf("%d", &n);
printf("Enter adjacency matrix (0 for no edge):\n");
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
scanf("%d", &graph[i][j]);
}
}
printf("Enter source vertex (0 to %d): ", n - 1);
scanf("%d", &source);
dijkstra(graph, n, source);
return 0;
}
```

**Output:**


C:\Users\BMSCECSE-L3-40\De
Enter number of vertices: 3
Enter adjacency matrix (0 for no edge):
0
1
4
0
0
2
0
0
0
Enter source vertex (0 to 2): 0
Vertex Distance from Source
0 0
1 1
2 3
Process returned 0 (0x0) execution time : 53.414 s
Press any key to continue.

## Lab program 11:

**Complete program in C**

```
#include <stdio.h>
#include <stdlib.h>
int board[20], n;
int place(int row, int col)
{
   for(int i = 1; i < row; i++)
   {
      if(board[i] == col ||
         abs(board[i] - col) == abs(i - row))
          return 0;
   }
   return 1;
}
void nQueens(int row)
{
   for(int col = 1; col <= n; col++)
   {
      if(place(row, col))
      {
         board[row] = col;
         if(row == n)
         {
            printf("\nSolution:\n");
            for(int i = 1; i <= n; i++)
               printf("%d ", board[i]);
            printf("\n");
         }
```

```
            else
            {
                nQueens(row + 1);
            }
        }
    }
}
int main()
{
    printf("Enter N: ");
    scanf("%d", &n);
    nQueens(1);
    return 0;
}
```

**Output:**

